

FinQuest Backend Architecture & Technology Choices

A Detailed Technical Document

Table of Contents

1. Introduction
 2. Project Context & Goals
 3. Backend Architecture Overview
 4. Framework: FastAPI
 5. Database: SQLAlchemy 2.0 + SQLite
 6. Authentication Stack
 7. Data Validation: Pydantic v2
 8. Configuration Management
 9. Middleware Layer
 10. API Design & Response Standardization
 11. Gamification Engine Design
 12. Services Layer Pattern
 13. Frontend Stack & Integration
 14. Security Architecture
 15. Deployment Strategy
 16. Trade-offs & Future Roadmap
 17. Conclusion
-

1. Introduction

FinQuest is a gamified personal financial management system. While the user interface presents charts, progress bars, and achievement badges, the real complexity lives in the backend: a carefully designed API that handles authentication, financial transactions, real-time analytics, streak tracking, and gamification logic.

This document explains the backend in detail. For every framework, library, and pattern, we discuss what it is, why it was chosen, and what problem it solves in the context of FinQuest.

2. Project Context & Goals

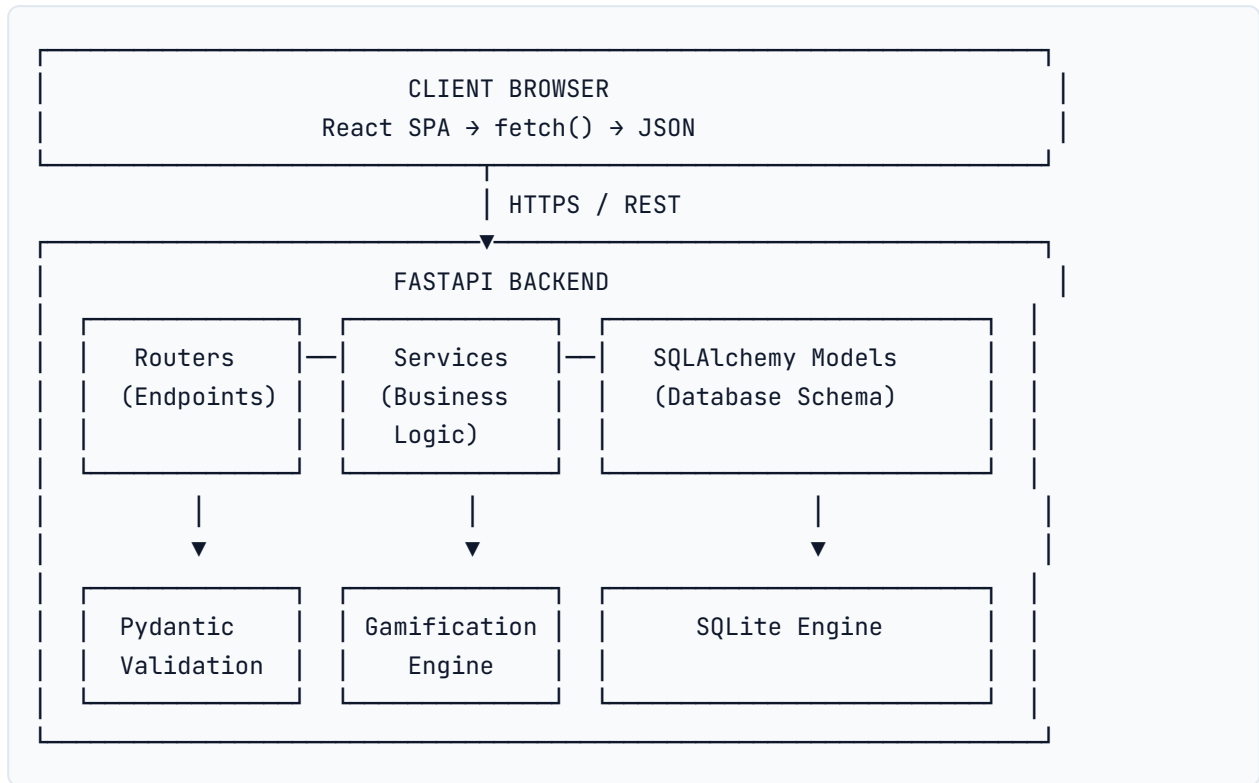
Before selecting technologies, we defined the project's non-functional requirements:

Requirement	Why It Mattered
Rapid Development	Final-year project with limited time; needed to ship features fast
Type Safety	Financial data must be validated strictly; bugs are costly
Easy Deployment	Should run on a single machine or small VPS without complex infrastructure
Modern API Standards	RESTful design with automatic documentation
Gamification Support	XP, levels, achievements, and streaks must be computed reliably
Security	Passwords, tokens, and financial data must be protected
Frontend Flexibility	SPA frontend consuming JSON APIs

These requirements directly shaped the technology choices.

3. Backend Architecture Overview

3.1 High-Level Structure



3.2 Layer Responsibilities

Layer	Responsibility	Example
Routers	Define URL endpoints and HTTP methods	<code>POST /api/v1/transactions</code>
Pydantic Schemas	Validate incoming/outgoing data	<code>TransactionCreate</code> , <code>BudgetCreate</code>
Services	Encapsulate business rules	<code>process_gamification_event()</code>
Models	Map Python classes to database tables	<code>User</code> , <code>Transaction</code> , <code>Budget</code>
Database	Persist data reliably	SQLite via SQLAlchemy engine

4. Framework: FastAPI

4.1 What Is FastAPI?

FastAPI is a modern, high-performance Python web framework for building APIs. It is built on top of Starlette (for async handling) and Pydantic (for data validation).

4.2 Why FastAPI Was Chosen

Reason 1: Automatic API Documentation

FastAPI automatically generates interactive OpenAPI documentation at `/docs` and `/redoc`. This is invaluable during development because:

- Frontend developers can test endpoints without writing curl commands
- API contracts are always up to date
- No separate documentation maintenance is required

Reason 2: Native Pydantic Integration

FinQuest handles money, dates, and user credentials. Data validation is non-negotiable. FastAPI uses Pydantic models natively:

```
class TransactionCreate(BaseModel):
    type: str = Field(..., pattern="^(income|expense)$")
    amount: Decimal = Field(..., gt=0, decimal_places=2)
    category_id: int
    transaction_date: date
```

This single model both documents and validates the API contract.

Reason 3: Dependency Injection

FastAPI's `Depends()` makes clean architecture easy:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/api/v1/users/me")
def get_me(current_user: User = Depends(get_current_active_user)):
    return current_user
```

The database session and authenticated user are injected where needed, reducing boilerplate and improving testability.

Reason 4: Performance

FastAPI is one of the fastest Python frameworks available. While FinQuest is not yet handling millions of requests, choosing a performant framework means we won't outgrow it quickly.

Reason 5: Async Support

Built on Starlette and ASGI, FastAPI supports asynchronous endpoints. This is useful for:

- Future AI insight generation
- Background notification processing
- External API calls (currency exchange rates)

Alternatives Considered

Framework	Why Not Chosen
Django	Too heavy; includes admin, forms, templating we don't need
Flask	Requires manual validation, routing, and documentation setup
Hono + Node.js	Initially considered, but Python's data ecosystem is stronger for analytics

5. Database: SQLAlchemy 2.0 + SQLite

5.1 What Is SQLAlchemy 2.0?

SQLAlchemy is the most widely used Python SQL toolkit and ORM. Version 2.0 introduces a new declarative mapping style using `Mapped` and `mapped_column`.

5.2 Why SQLAlchemy 2.0 Was Chosen

Reason 1: Type Safety with Mapped Columns

SQLAlchemy 2.0 offers excellent type annotations:

```
class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True)
    current_xp: Mapped[int] = mapped_column(Integer, default=0)
```

This gives us IDE autocomplete, static type checking, and fewer runtime errors.

Reason 2: Declarative Models

Tables are defined as Python classes. Relationships are expressed naturally:

```
transactions: Mapped[List["Transaction"]] = relationship(back_populates="user",
lazy="dynamic")
```

This makes the codebase self-documenting.

Reason 3: Database Agnosticism

SQLAlchemy abstracts PostgreSQL, MySQL, and SQLite behind the same API. FinQuest ships with SQLite by default, but moving to PostgreSQL only requires changing the `DATABASE_URL` environment variable.

5.3 Why SQLite?

Reason 1: Zero Configuration

SQLite requires no server installation. For a final-year project that needs to run immediately on any machine, this is ideal.

Reason 2: Sufficient for the Workload

SQLite handles:

- Single-user to low-concurrent-user scenarios
- Financial transaction records in the thousands
- Complex joins for analytics

This matches FinQuest's current scope.

Reason 3: Portability

The entire database is a single file (`finquest.db`), making backups and demos trivial.

When to Upgrade

If FinQuest grows to many concurrent users or requires high write throughput, we would migrate to PostgreSQL without rewriting application code.

6. Authentication Stack

6.1 Components

Component	Library	Purpose
Password Hashing	<code>passlib</code> + <code>bcrypt</code>	Store passwords securely
JWT Tokens	<code>python-jose</code>	Generate and verify access/refresh tokens
Token Storage	SQLite <code>refresh_tokens</code> table	Track and revoke refresh tokens
Cookie Security	FastAPI <code>Response.set_cookie</code>	HttpOnly cookie for access token

6.2 Why JWT?

JWT (JSON Web Tokens) were chosen because they are:

- **Stateless:** The server does not need to query a session store for every request
- **Self-contained:** Tokens carry user identity and expiry information
- **Industry Standard:** Widely understood and supported

6.3 Why Refresh Token Rotation?

FinQuest uses two tokens:

- **Access Token:** Short-lived (30 minutes), used for API calls
- **Refresh Token:** Long-lived (7 days), used to obtain new access tokens

Refresh token rotation means every refresh invalidates the old token. This prevents:

- Replay attacks using stolen refresh tokens
- Long-lived compromise windows

6.4 Why bcrypt?

bcrypt is the de-facto standard for password hashing because it is:

- **Slow:** Resistant to brute-force attacks
- **Adaptive:** Work factor can be increased as hardware improves
- **Well-tested:** Decades of real-world security validation

We use 12 rounds of bcrypt, which provides a good balance between security and performance.

7. Data Validation: Pydantic v2

7.1 What Is Pydantic?

Pydantic is a data validation library that uses Python type hints to enforce schemas at runtime.

7.2 Why Pydantic v2?

Reason 1: Runtime Validation from Type Hints

Developers write normal Python classes, and Pydantic enforces the types:

```
class BudgetCreate(BaseModel):
    name: str = Field(..., min_length=1, max_length=100)
    amount: Decimal = Field(..., gt=0, decimal_places=2)
    period_start: date
    period_end: date
```

If the frontend sends a negative amount or a date in the wrong format, Pydantic rejects it automatically.

Reason 2: Automatic Error Messages

When validation fails, Pydantic returns clear error messages:

```
{
  "detail": [
    {
      "loc": ["body", "amount"],
      "msg": "ensure this value is greater than 0",
      "type": "value_error.number.not_gt"
    }
  ]
}
```

This makes debugging frontend-backend integration easier.

Reason 3: Serialization

Pydantic models convert SQLAlchemy objects to JSON-ready dictionaries:

```
return UserResponse.model_validate(user)
```

This eliminates manual field mapping and reduces bugs.

8. Configuration Management

8.1 Pydantic Settings

Configuration is managed through `pydantic-settings`, which loads values from environment variables with sensible defaults:

```
class Settings(BaseSettings):
    SECRET_KEY: str = Field(default_factory=lambda: secrets.token_hex(32))
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 30
    REFRESH_TOKEN_EXPIRE_DAYS: int = 7
    DATABASE_URL: str = "sqlite:///./finquest.db"
```

8.2 Why Pydantic Settings?

- **Type Safety:** Configuration values are validated at startup
- **Environment Variables:** Sensitive values like `SECRET_KEY` are read from the environment, not hardcoded
- **Sensible Defaults:** The app runs out of the box in development mode
- **Documentation:** The settings class serves as living configuration documentation

9. Middleware Layer

9.1 What Is Middleware?

Middleware are functions that process every incoming request and outgoing response. FinQuest uses three custom middleware components.

9.2 CORS Middleware

Purpose: Allows the frontend (running on `localhost:3000` or `localhost:5173`) to call the backend.

Why It's Needed: During development, the frontend and backend run on different ports, which triggers browser CORS policies. The CORS middleware whitelists trusted origins.

9.3 RateLimitMiddleware

Purpose: Prevents abuse by limiting requests per IP address.

Implementation: In-memory store tracking request timestamps. Stricter limits are applied to auth endpoints (5 requests per 15 minutes) compared to general endpoints (100 requests per 60 seconds).

Why It's Critical: Protects against brute-force login attempts and accidental DoS from rapid UI interactions.

9.4 SecurityHeadersMiddleware

Adds security headers to every response:

- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY
- X-XSS-Protection: 1; mode=block
- Referrer-Policy: strict-origin-when-cross-origin
- Strict-Transport-Security (production only)

Why It Matters: These headers protect against common attacks like clickjacking, MIME sniffing, and XSS.

9.5 Request Metadata Middleware

Adds X-Request-ID and X-Response-Time-Ms headers to every response.

Why It Matters: Makes debugging and performance monitoring possible.

10. API Design & Response Standardization

10.1 Generic API Response Wrapper

Every endpoint returns the same envelope:

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "timestamp": "2026-05-08T20:00:00",
    "request_id": "uuid",
    "pagination": { ... }
  }
}
```

10.2 Why Standardize Responses?

- **Predictable Frontend Code:** The frontend always checks `response.data`

- **Consistent Error Handling:** Errors use the same envelope with `success: false`
- **Metadata:** Pagination and request IDs are available on every call

10.3 Pagination

List endpoints use `PaginatedResponse[T]` :

```
{
  "items": [ ... ],
  "total": 100,
  "page": 1,
  "pages": 5
}
```

This supports large transaction histories without overwhelming the frontend.

11. Gamification Engine Design

11.1 Why a Dedicated Service?

Gamification rules are complex and cross-cutting. Putting them in a dedicated service (`gamification_service.py`) keeps routers clean and makes the rules testable in isolation.

11.2 Core Concepts

Concept	Implementation
XP	Integer points stored on <code>User.total_xp_earned</code>
Level	<code>floor(total_xp_earned / 100) + 1</code>
Streaks	<code>Streak</code> model tracks consecutive daily logins
Achievements	<code>Achievement</code> definitions + <code>UserAchievement</code> unlock records

11.3 Event-Driven Processing

When a user performs an action, `process_gamification_event()` is called:

```
def process_gamification_event(db, user_id, event_type, metadata=None):
    # Update streaks
    # Award XP based on event type
    # Check for new achievements
    # Return GamificationDelta
```

This centralizes all gamification logic and makes it easy to add new event types.

11.4 Why This Design?

- **Single Source of Truth:** All XP/level calculations happen in one place
- **Transaction Safety:** Database commits happen atomically with financial records
- **Frontend Feedback:** The `GamificationDelta` tells the UI exactly what changed

12. Services Layer Pattern

12.1 What Is the Services Layer?

The services layer sits between routers and models. It contains business logic that doesn't belong in HTTP handlers or database models.

12.2 Examples

Service	Responsibility
<code>gamification_service.py</code>	XP, levels, streaks, achievements
<code>export_service.py</code>	Format data for CSV, JSON, Excel export
<code>notification_service.py</code>	Generate user notifications

12.3 Why This Pattern?

- **Separation of Concerns:** Routers handle HTTP; services handle business rules
- **Reusability:** The same service can be called from multiple endpoints
- **Testability:** Services can be unit tested without HTTP scaffolding

13. Frontend Stack & Integration

13.1 Why React 19?

React 19 provides:

- Modern concurrent features
- Improved performance
- Strong ecosystem and community support
- Excellent TypeScript integration

13.2 Why Vite?

Vite was chosen over Create React App because it offers:

- Faster development server startup
- Instant Hot Module Replacement (HMR)
- Optimized production builds
- Native TypeScript support

13.3 Why TanStack Query?

TanStack Query (React Query) handles:

- Caching server data
- Background refetching
- Mutation invalidation
- Loading and error states

This eliminates significant boilerplate compared to raw `useEffect` + `fetch`.

13.4 Why Tailwind CSS?

Tailwind enables rapid UI development with utility classes. Combined with shadcn/ui primitives, we built a polished interface quickly without writing custom CSS from scratch.

13.5 Frontend-Backend Contract

The frontend communicates with the backend through a typed REST API:

```
// services/api.ts
export const transactionApi = {
  create: (data: { type: string; amount: string | number; category_id: number;
transaction_date: string }) =>
    api.post("/transactions", data),
}
```

TypeScript ensures the frontend sends exactly what the backend expects.

14. Security Architecture

14.1 Defense in Depth

FinQuest employs multiple security layers:

Layer	Mechanism
Transport	HTTPS in production
Authentication	JWT access tokens + refresh token rotation
Passwords	bcrypt hashing
Session	HttpOnly cookies
Rate Limiting	Per-IP request throttling
Headers	Security headers on all responses
Input Validation	Pydantic strict validation
Output Safety	React auto-escapes XSS
SQL Injection	SQLAlchemy ORM parameterized queries

14.2 Why This Matters for a Finance App

Even though FinQuest is a student project, financial data is sensitive. Building security in from the start:

- Demonstrates professional development practices
 - Protects demo users' data
 - Provides a foundation for production deployment
-

15. Deployment Strategy

15.1 Development Mode

```
# Backend
cd app/backend
source venv/bin/activate
uvicorn app.main:app --host 127.0.0.1 --port 8001

# Frontend
cd app
npm run dev
```

Vite proxies `/api` requests to the backend.

15.2 Production Mode

```
cd app
npm run build
cd backend
source venv/bin/activate
uvicorn app.main:app --host 0.0.0.0 --port 8001
```

The backend serves the built frontend from `dist/public/` and handles SPA routing.

15.3 Why Single-Server Deployment?

For a final-year project, a single-server deployment is:

- **Simple:** One process handles both frontend and backend
 - **Cost-effective:** Runs on a small VPS or free-tier hosting
 - **Portable:** Easy to demo and submit
-

16. Trade-offs & Future Roadmap

16.1 Conscious Trade-offs

Decision	Trade-off	Mitigation
SQLite	Limited concurrency	Easy migration path to PostgreSQL
In-memory rate limiting	Doesn't scale across multiple servers	Use Redis in production
Synchronous endpoints	Simpler but blocks	Most endpoints are fast; async available when needed
Single-server deployment	Not horizontally scalable	Docker + load balancer for future scaling

16.2 Planned Improvements

Feature	Description
PostgreSQL Migration	For production multi-user scenarios
Redis	Distributed rate limiting and caching
Celery	Background task processing for reports and notifications
Docker	Containerized deployment
Comprehensive Tests	Backend pytest suite and frontend Vitest suite
AI Insights	Spending pattern analysis using scikit-learn
Multi-currency	Support for EUR, GBP, NGN

17. Conclusion

FinQuest's backend is designed around simplicity, type safety, and clean architecture. FastAPI provides a modern API foundation, SQLAlchemy handles data persistence, Pydantic ensures data integrity, and custom middleware provides security and observability.

Every technology choice was made with the project's context in mind: a final-year project that needs to be functional, secure, and demonstrable without over-engineering. The architecture supports the current feature set while remaining extensible for future enhancements.

Document generated: May 2026

FinQuest v1.0.0 — Backend Architecture & Technology Choices